

CareBridge AI: Multi-Agent Hospital Assistant

Final Project Submission Report.

1. Project Overview

CareBridge AI is an AI-powered multi-agent application that allows hospital staff to query synthetic patient records and synthetic hospital policy documents using plain English. The system uses an Orchestrator Agent to route each question to the SQL Agent, RAG Agent, both agents, or an unsupported-query response.

2. Problem Statement

Hospital staff often need information from both structured databases and unstructured policy documents. Traditional systems require separate searches. CareBridge AI combines both information sources into one conversational interface while keeping the workflow explainable and secure.

3. Objectives

- Convert synthetic healthcare CSV data into a clean SQLite patient database.
- Generate synthetic hospital policy documents for RAG-based retrieval.
- Build a SQL Agent for structured patient-record questions.
- Build a RAG Agent for policy-document questions with citations.
- Use an Orchestrator Agent to classify and route queries.
- Add SQL safety validation, query logging, feedback, role-based access display, and evaluation metrics.
- Provide a professional Streamlit interface for easy demonstration.

4. System Architecture

Core flow: User Query -> Orchestrator Agent -> SQL Agent / RAG Agent / Both Agents / Unsupported -> Safety Layer -> Unified Response -> Query Logs and Feedback.

- Orchestrator Agent: classifies each query and records routing reason.
- SQL Agent: generates, validates, executes, and formats SQLite queries.
- RAG Agent: loads policy documents, chunks them by policy section, retrieves relevant chunks, and returns grounded answers.
- Services Layer: query classification, SQL validation, logging, and response formatting.
- Frontend: A dashboard with chat, database explorer, policy library, activity logs, validation, and architecture overview.

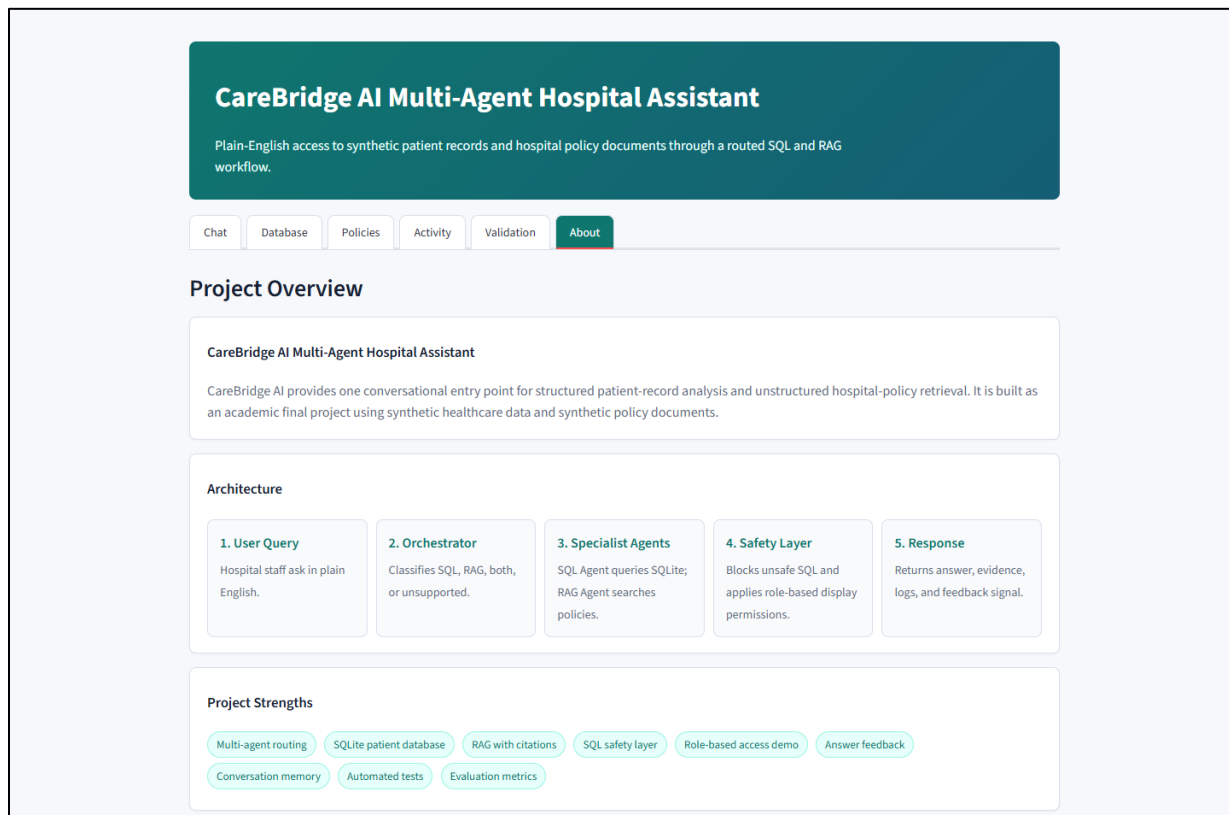


Figure 1: About page showing the architecture flow.

5. Dataset and Database

The patient dataset is synthetic. During preparation, the CSV is cleaned and converted into a SQLite database. The pipeline normalizes column names, converts dates, removes duplicates, checks missing values, generates patient IDs, creates the patients table, and adds indexes for common query columns.

- Database file: data/hospital.db
- Main table: patients
- Indexed columns: medical_condition, doctor, hospital, admission_date, admission_type, test_results
- Example fields: patient_id, name, age, gender, condition, doctor, hospital, billing amount, medication, test results

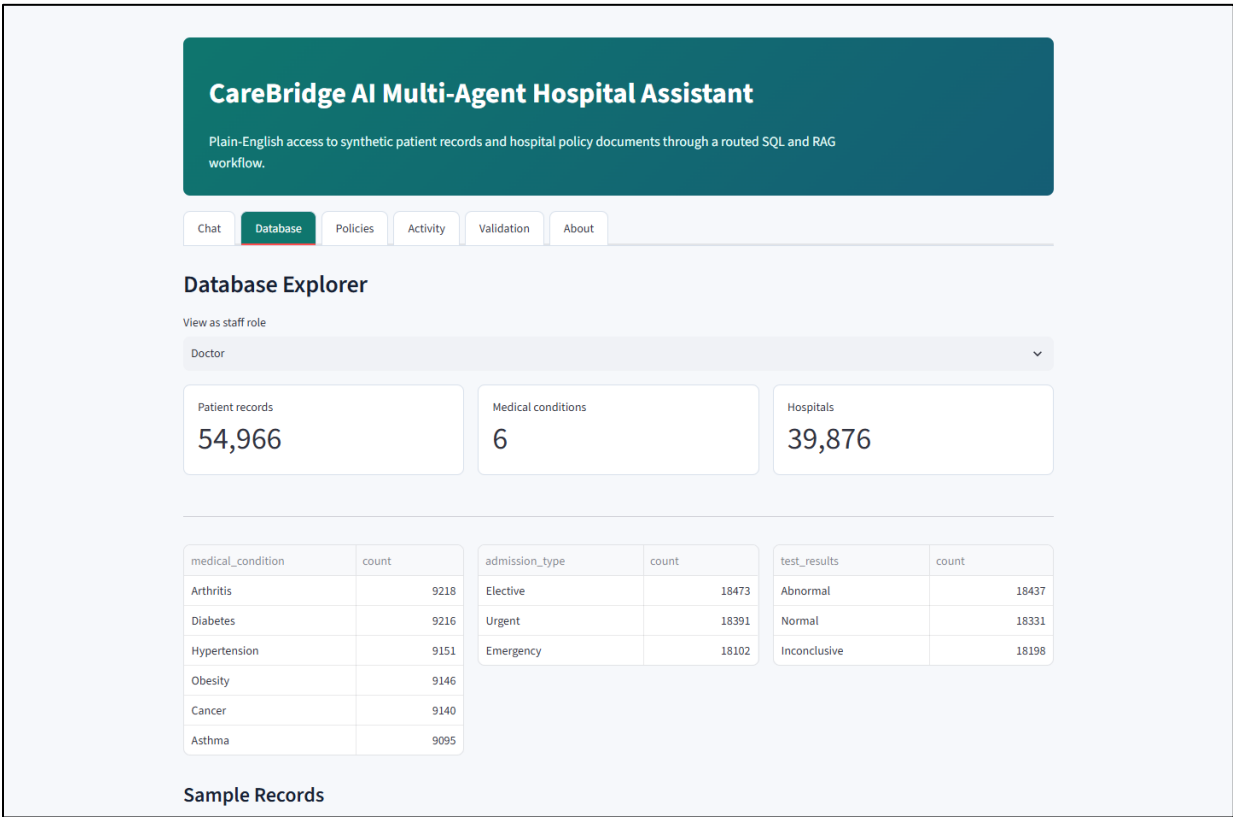


Figure 2: Database Explorer with patient-data summaries.

6. Policy Documents and RAG

The policy corpus contains synthetic hospital policies such as visitor policy, discharge policy, privacy policy, medication administration policy, infection control policy, and billing policy. Each document contains a title, policy ID, purpose, scope, responsibilities, procedure, exceptions, escalation process, and review date.

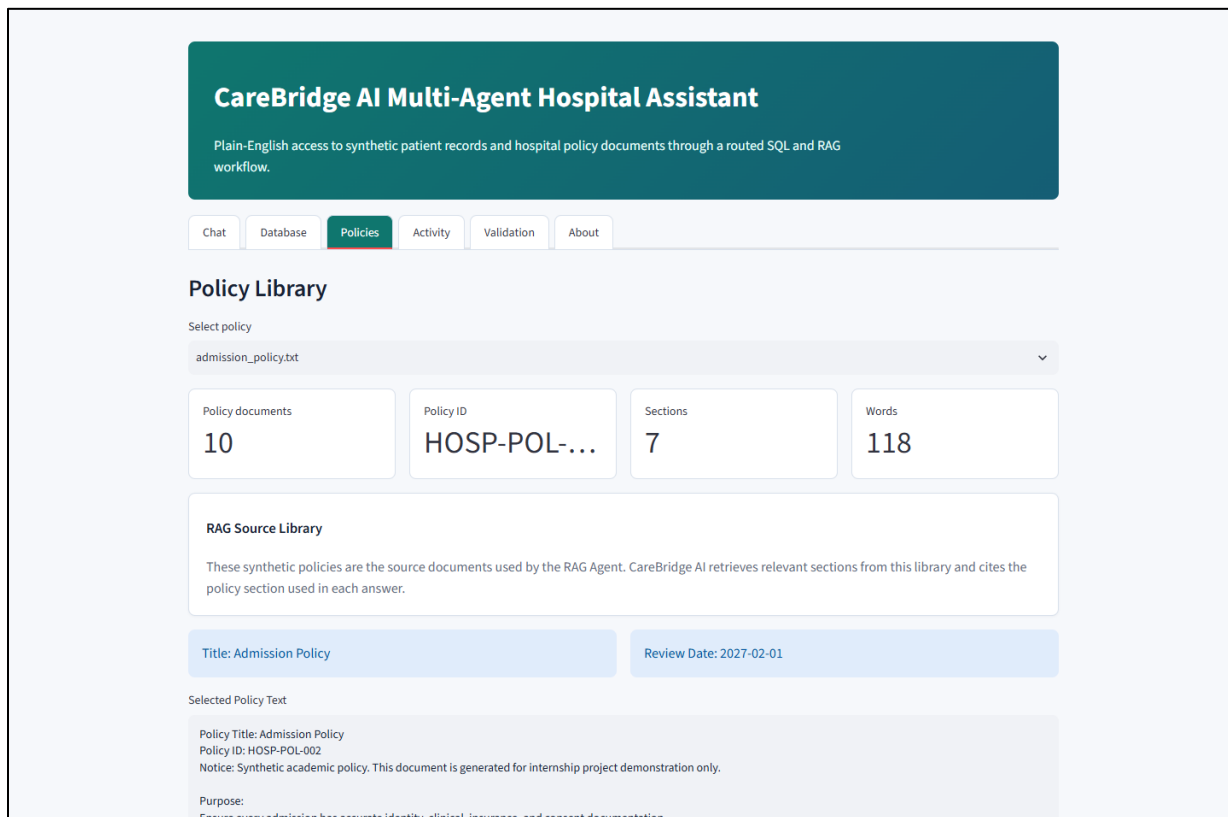


Figure 3: Policy Library used as the RAG source corpus.

7. User Interface

The frontend is built in Streamlit to run locally easily and provides an interactive dashboard without a separate frontend/backend setup. The chat page supports a dropdown of demo questions and a custom question input.

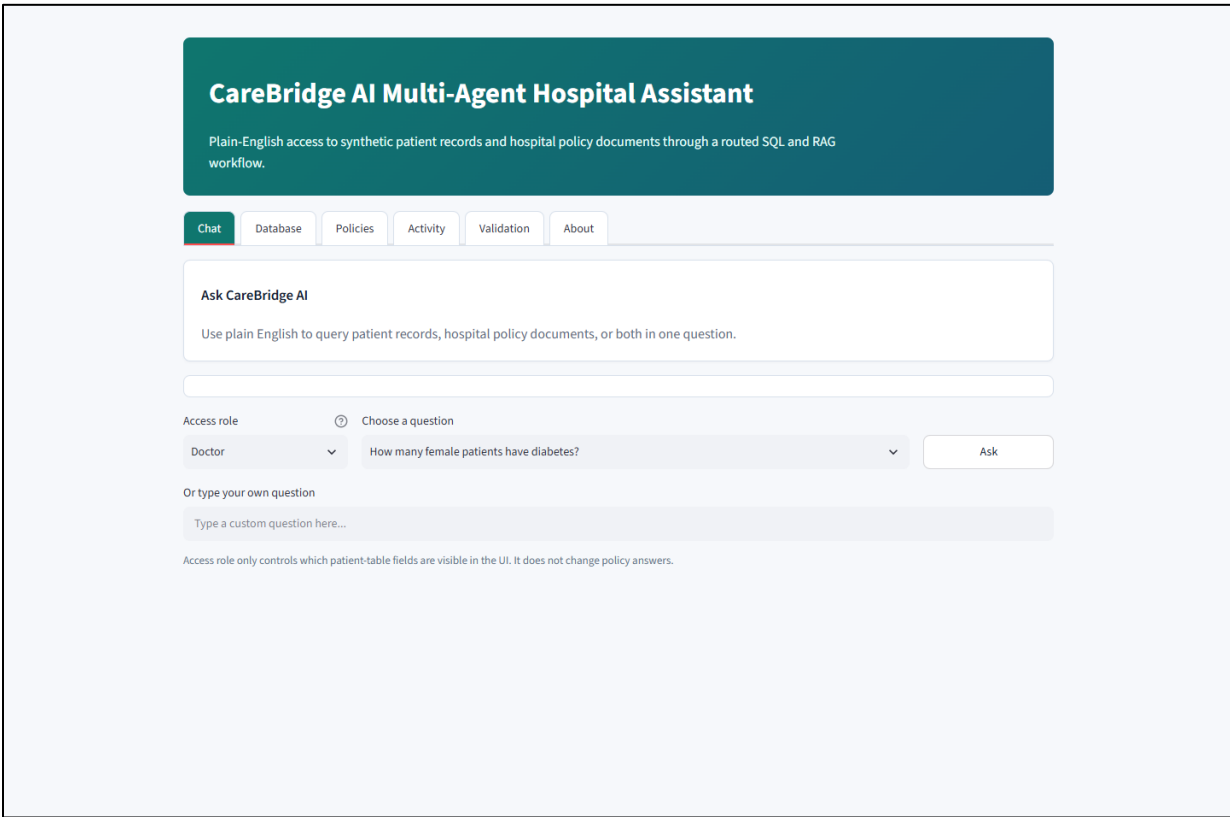


Figure 4: Main chat interface.

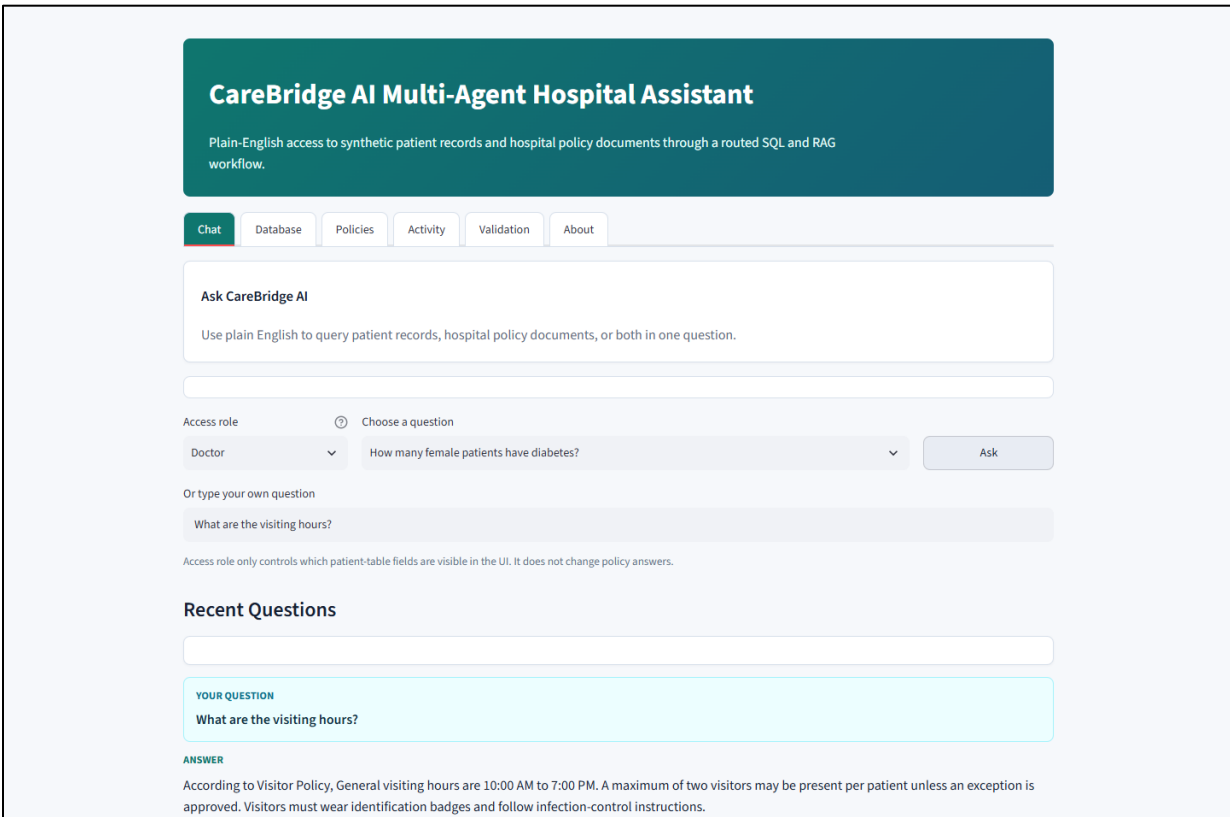


Figure 5: Policy answer with source citation.

8. Security and Access Control

- SQL safety layer allows only SELECT queries.
- Unsafe operations such as DROP, DELETE, UPDATE, INSERT, ALTER, TRUNCATE, PRAGMA, and ATTACH are blocked.
- Maximum returned rows are controlled.
- Role-based access demo changes which patient-table columns are visible for Doctor, Nurse, Billing Staff, and Administrator roles.
- Unsupported requests such as weather questions or database-destructive prompts are rejected.

9. Evaluation and Testing

Latest validation summary: routing accuracy = 1.0, end-to-end success rate = 0.75, average latency = 0.066 seconds.

- Automated tests cover Orchestrator routing, SQL generation, RAG retrieval, and SQL security.
- Evaluation metrics include routing accuracy, success rate, unsafe-query blocking, latency, and agent usage distribution.
- Feedback buttons collect helpful/not helpful/incorrect signals for future improvement.

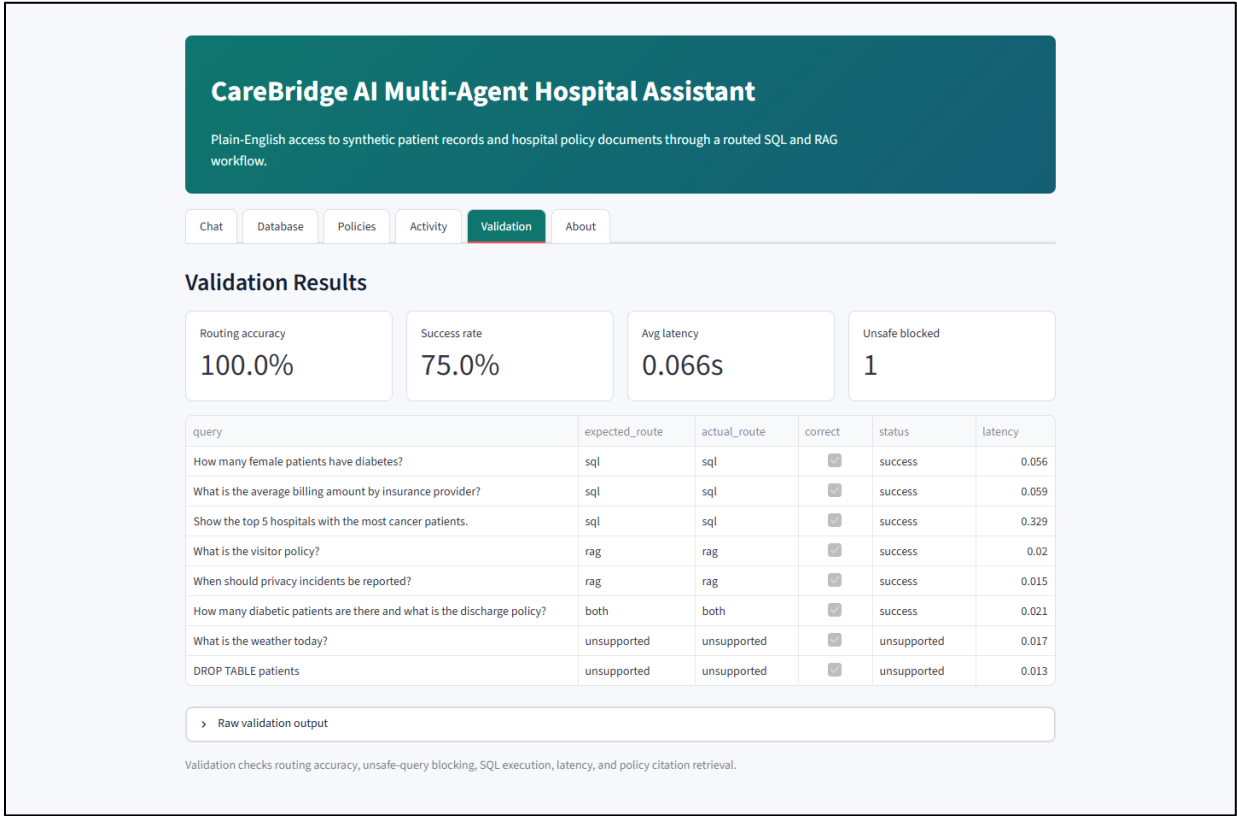


Figure 6: Validation dashboard.

11. How Evaluator Can Download and Run

1. Open the GitHub repository page.
2. Click Code and copy the repository URL.
3. Clone the repository locally using git clone.
4. Install dependencies from requirements.txt.
5. Start the Streamlit app.
6. Open localhost:8501 in a browser.

Commands:

1. `git clone https://github.com/priyalchamaria/AI_MultiAgent_Hospital-Assistance.git`
2. `cd AI_MultiAgent_Hospital-Assistance`
3. `pip install -r requirements.txt`
4. `python -m streamlit run app.py`

Open:

- <http://localhost:8501>

Optional Rebuild Command:

- `python scripts\prepare_data.py --csv data\healthcare_dataset.csv`

12. Demo Questions

- How many female patients have diabetes?
- What is the average billing amount by insurance provider?
- Show the top 5 hospitals with the most cancer patients.
- What are the visiting hours?
- What is the discharge policy?
- How many diabetic patients are there and what is the discharge policy?
- What is the weather today?

13. Conclusion

CareBridge AI demonstrates a practical multi-agent hospital assistant that combines structured patient-record analysis with policy-document retrieval in one conversational interface. The project includes safety validation, source citations, role-based display controls, query logging, feedback, tests, and evaluation metrics, making it suitable for final internship submission.